

CSAFD401

EMBEDDED SYSTEM FIRMWARE DEVELOPMENT

Develop Embedded System Firmware

Competence

RQF Level: 4

Learning Hours



80

Credits: 8

Sector: ICT and Multimedia

Trade: Computer System and Architecture

Module Type: Specific

Curriculum: ICTCSA4001 TVET Certificate IV in Computer System and Architecture

Copyright: © Rwanda TVET Board, 2023

1200

Issue Date: September, 2023

Purpose statement	This specific module describes the skills, knowledge and attitude required to develop Firmware for embedded Systems. This module is intended to prepare student pursuing TVET Level 4 in computer system and architecture. Upon completion of this module, the learner will be able to Identify Firmware requirement, Design Firmware Architecture, implement firmware system design and Deploy firmware					
Learning assumed to be in place	▪ Develop Embedded System Hardware, ▪ Apply C++ programming fundamentals, ▪ Apply C programming fundamentals, ▪ Apply Digital Electronics					
Delivery modality	Training delivery		100%	Assessment		Total 100%
	Theoretical content		20%	Formative assessment	20%	50%
	Practical work:		80%		80%	
	Group project and presentation	30%				
	Individual project /Work	50%				
			Summative Assessment		50%	

Elements of Competence and Performance Criteria

Elements of competence	Performance criteria
1. Identify Firmware requirement.	1.1 Data are properly collected according to the embedded system requirements
	1.2 Data are properly analyzed based on embedded system requirements
	1.3 Firmware requirements are properly extracted according to the data analyzed
2. Design Firmware Architecture	2.1 Tools, materials and equipment are correctly selected according to the firmware requirements
	2.2 Drawing environment is properly prepared based on firmware requirements
	2.3 Firmware architecture diagrams are properly drawn based on hardware architecture
3. Implement firmware system design	3.1 Programming languages are properly selected according to the requirements of firmware system
	3.2 Development environment is correctly prepared based on programming language.
	3.3 Communication protocols are properly selected based on communication interfaces
	3.4 Memory is perfectly managed according to the performance requirement
	3.5 Firmware drivers' source codes are properly written based on the embedded system requirements
4. Deploy firmware	4.1 Deployment environment is properly prepared according to the distributed firmware package and hardware architecture
	4.2 Firmware is effectively tested according to the embedded system

	requirements
	4.3 Firmware Image is correctly exported in line with the deployment requirement
	4.4 Firmware is properly documented according to the work done

Intended knowledge, skills and attitude

Knowledge	Skills	Attitude
✓ Define Embedded System Firmware ✓ Describe basic electronics principles ✓ Identify version control systems ✓ Describe debugging techniques and tools concepts	✓ Analyze embedded system firmware ✓ Test embedded system firmware ✓ Programming skills ✓ Track changes of firmware ✓ Debug embedded system firmware ✓ Configure and interfacing a hardware peripheral ✓ Security ✓ Elaborate technical report	✓ Attentive ✓ Problem solving ✓ Critical thinking ✓ Team work ✓ Hardworking ✓ Creative ✓ Time management ✓ Decision making ✓ Reasoning ✓ Adaptability

Course content

Learning outcomes	At the end of the module the learner will be able to: 1. Identify Firmware requirement 2. Design Firmware Architecture 3. Implement firmware system design 4. Deploy firmware
--------------------------	--

Learning outcome 1: Identify Firmware requirement	Learning hours: 20
Indicative content	
• Collection of Data for firmware development ✓ Description of Firmware + Definition + Types of Firmware Data + key components of firmware architecture + Applications of firmware + Firmware development stage/process ✓ Data collection methods • Analyzing collected data ✓ Hardware requirement + Processing Power + Power design + Communication and Interfaces + Storage capacity ✓ Embedded system requirements	

-  Functional
-  Performance
-  Power
-  Environmental
-  Form Factor and Size Analysis
-  Communication and Interface Analysis
-  Analog and Digital Input/Output Analysis

- **Extraction of Firmware requirements**

- ✓ Identification of system usage

-  Functional requirements

-  Non-functional requirements

- ✓ System feasibility

-  Technical

-  Economic

-  Legal

-  Operational

-  Scheduling

Resources required for the learning outcome

Equipment	▪ Computer
Materials	▪ Internet ▪ Electricity
Tools	▪ Web browsers ▪ Text editor
Facilitation techniques or Learning activity	▪ Demonstration and simulation ▪ Practical exercise ▪ Individual work ▪ Trainer-guided

	▪ Group discussion
Formative assessment methods /(CAT)	▪ Written assessment ▪ Oral presentation ▪ Performance assessment

Learning outcome 2: Design Firmware Architecture	Learning hours: 10
Indicative content	
• Selection of Tools, materials, and equipment ✓ Tools ✓ Materials ✓ Equipment • Preparation of drawing environment ✓ Installation of a Diagramming Tool ✓ Identification of Symbols and Notation ✓ Gathering Complete Hardware Information  Microcontroller/Processor  Hardware Components ✓ Drawing Templates ✓ Applying Color Scheme and Styling • Drawing Firmware architecture diagrams ✓ Identification of Firmware Modules. ✓ Diagram Types ✓ Create Hardware Block Diagram ✓ Create The Functional Flow Diagram(flowchart) ✓ Represent Firmware Modules. ✓ Create an architecture Diagram. ✓ Hardware Interaction ✓ Data Flow and Communication (Sequence Diagram)	

- Documentation of firmware architecture

Resources required for the indicative content

Equipment	▪ Computer, ▪ White board
Materials	▪ Internet ▪ Notebooks
Tools	▪ Microsoft Visio ▪ Lucidchart ▪ Draw.io ▪ Edraw Max ▪ Smart Draw ▪ Adobe Illustrator ▪ Text Editor ▪ Hand Drawing Tools
Facilitation techniques or Learning activity	▪ Demonstration ▪ Group discussion ▪ Practical exercise ▪ Trainer guided
Formative assessment methods /(CAT)	▪ Written assessment ▪ Oral presentation ▪ Practical Assessments

Indicative content

- **Identification of programming Languages used for firmware development**

- ✓ Types
- ✓ Advantages and disadvantages
- ✓ Comparison of Languages for firmware development

- **Preparation of Development Environment**

- ✓ Identification of Integrated Development Environment IDE
 -  Definition
 -  Types
 -  Features
- ✓ Installation of Integrated Development Environment (IDE)
- ✓ Configuration of IDE

- **Selection of communication protocols**

- ✓ Definition
 -  Protocols
 -  Communication protocols
- ✓ Importance of communication protocols
- ✓ Types of communication protocols
 -  Universal Asynchronous Receiver Transmitter (UART) protocols
 -  Serial Peripheral Interface (SPI) protocols
 -  Intern-Integrated Circuit (I2C) protocols
 -  Universal Serial Bus (USB) protocols

- **Identification of default segments of data memory**

- ✓ .data
- ✓ .bss
- ✓ Heap
- ✓ Stack

- **Performance of memory management in Embedded C**

- ✓ Static Memory Allocation
- ✓ Dynamic Memory Allocation

- **Description of Concepts for Developing Firmware**

- ✓ Embedded C Programming concepts

-  Data types

-  Structures and Unions

-  Bit fields

-  Preprocessor directives

- ✓ Application of Application Programming Interfaces (APIs) and Hardware Abstraction Layers (HALs) fundamentals

-  APIs versus HALs

-  The API and HAL Landscape

-  The API Scope

-  API Characteristics

-  Designing your own API

- ✓ Identification of HAL design process

-  HAL Design for GPIO

-  HAL Design for SPI

-  HAL design for EEPROM and Memory devices

- **Implementation firmware modules**

- ✓ Initialization Module
- ✓ Data Acquisition Module
- ✓ Data Processing Module
- ✓ Control Module
- ✓ Communication Module

- ✓ Memory Management Module
- ✓ Timer Module
- ✓ Interrupt Handler Module
- ✓ Power Management Module
- ✓ User Interface Module
- ✓ Error Handling Module
- ✓ Security Module
- ✓ Bootloader Module
- ✓ Diagnostic and Debugging Module
- **Writing drivers source code**
 - ✓ Identification of Driver Interfaces
 - ✓ Device Driver Models
 - ✓ Driver module definition
 - ✓ Encapsulation and data hiding
 - ✓ Abstract data types
 - ✓ Callback functions
 - ✓ Error Handling
 - ✓ Memory Mapping Methodologies
 -  Mapping Memory Directly
 -  Mapping Memory with Pointers
 -  Mapping Memory with Structures
 - ✓ Using Pointer arrays in Driver design

Resources required for the indicative content

Equipment	▪ Computer ▪ White Board
Materials	▪ Internet, ▪ Notebooks
Tools	▪ Microsoft Visio

	▪ Text Editors ▪ Arduino IDE
Facilitation techniques or Learning activity	▪ Demonstration ▪ Group discussion ▪ Practical exercise ▪ Trainer-guided
Formative assessment methods /(CAT)	▪ Written assessment ▪ Oral presentation ▪ Practical Assessments

Learning outcome 4: Deploy firmware	Learning hours: 10
Indicative content	
• Preparation of deployment environment ✓ Selection of deployment environment  Virtual  Physical ✓ Identification of firmware deployment tools • Testing the firmware ✓ Description of testing techniques  Identification of Techniques  Advantages and disadvantages  STLC (Software Testing Life Cycle) ✓ Implementation of Testing Techniques  Unit Test  Integration Test	

- **Exporting the firmware image**

- ✓ Description on .hex file
- ✓ Creation of hex file

-  Selection of target device

-  Exporting .hex file

- ✓ Flashing the .hex file to microcontroller

- **Documentation of the firmware**

- ✓ Firmware technical documentation

-  Selection of documentation tool

-  Installation of documentation tool

-  Documentation project setup

-  Comments fundamentals

-  Documenting enum and struct

-  Documenting functions

-  Documenting Modules

-  Creation of a reusable template

- ✓ Firmware user manual

Resources required for the indicative content

Equipment	▪ Computer ▪ White Board
Materials	▪ Internet ▪ Notebooks
Tools	▪ Microsoft Visio ▪ Text Editors ▪ Doxygen ▪ XLoader ▪ Natural Docs ▪ Sphinx ▪ Doxypress

	▪ Avrdude ▪ Arduino IDE ▪ PlatformIO ▪ Flip ▪ Teensy Loader
Facilitation techniques or Learning activity	▪ Demonstration ▪ Group discussion ▪ Practical exercise
Formative assessment methods /(CAT)	▪ Written assessment ▪ Oral presentation ▪ Practical Assessments

Integrated/Summative assessment

Integrated situation

ABC Electronics is a growing startup that specializes in creating home automation products. They are currently working on a new project to develop an environmental monitoring system that will be used to monitor indoor temperature and humidity. The company is facing a problem that they have the circuit design of this system but there is no embedded software to manage the functionality of this system hardware.

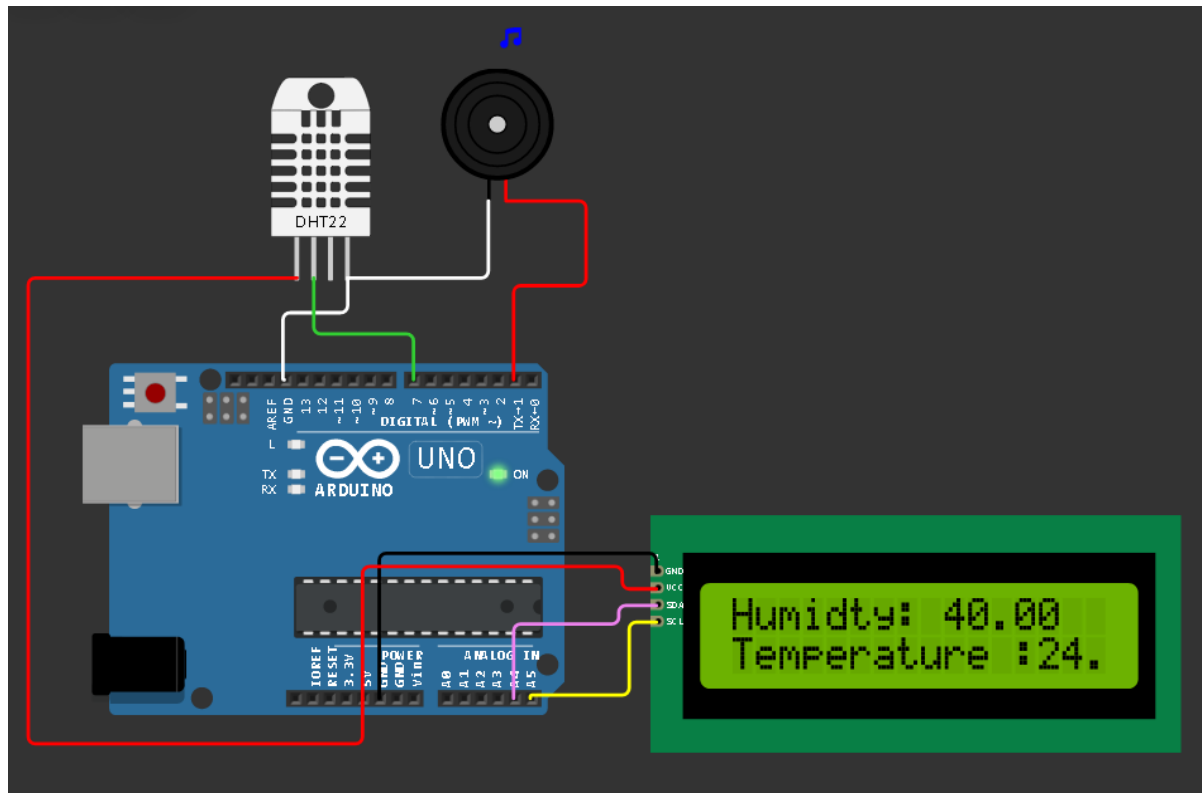
As a firmware developer, you are hired by ABC Electronics to develop a software that will solve this problem by being integrated into this system hardware to monitor the temperature and humidity levels in the clients' living spaces. The device will display real-time temperature and humidity data on the LCD screen and provide sound alerts when certain thresholds are crossed. The system will display temperature (°C) and humidity (%) data on the LCD screen in a user-friendly format. When the temperature and humidity rates out of normal conditions, the firmware should trigger audio alerts in the Buzzer and display warning messages.

Incorporate error handling routines to detect and manage sensor failures or communication issues effectively. Display appropriate error messages on the LCD to notify users of any problems.

After the work, prepare clear and concise documentation explaining how to use the device, interpret the data displayed, and generate the firmware image to be flashed in the system hardware.

Note: The normal or comfortable room temperature in Celsius degrees is generally considered to be between 20°C to 25°C (68°F to 77°F) and the recommended indoor relative humidity levels for comfort and health usually range between 40% to 60%.

The circuit diagram of the system is provided below:



The work must be performed within 6 hours (6).

All equipment, tools and materials are provided.

Resources

Tools	
	▪ Integrated Development Environment (IDE)

		▪ Microcontroller Programming Tools ▪ Firmware Update Tools ▪ Code Version Control (e.g., Git) ▪ Simulation and Emulation Tools ▪ Documentation Tools (Doxygen, Natural Docs, Sphinx, Doxypress,...) ▪ Debugging Tools ▪ Testing Frameworks ▪ Knowledge Resources (technical documentation, datasheets, forums, etc.)			
Equipment		▪ Personal Computer (PC) ▪ Hardware circuit			
Materials/ Consumables		▪ Internet Connectivity			
Assessable outcomes	Assessment criteria (Based on performance criteria)	Indicator	Observation		Marks allocation
			Yes	No	
Design Firmware Architecture (5%)	Selecting of Tools, materials and equipment	C or C++ Programming Language is selected			1
		IDE is selected			1
		Documenting tool is selected			1
Implement firmware system design. (68%)	Preparing of Development environment	IDE is installed			1
		IDE is configured			2
	Writing Firmware source codes	Temperature is displayed in °C			5
		Humidity is displayed in %			5
		High Temperature			5

		alert is configured			
		Low Temperature alert is configured			5
		High Humidity alert is configured			5
		Low Humidity alert is configured			5
		Buzzer ports are configured			4
		LDC ports are configured			4
Deploy Firmware (27%)	Preparation of Deployment environment	Flashing tool is installed			2
	Exporting Firmware Image	.hex file for the firmware is created			4
		The firmware is flashed			4
	Documentation of Firmware	Documentation tool is installed			1
		Documentation tool is configured			2
		Firmware is documented			3
Total marks		60			
Percentage Weightage		100%			
Minimum Passing line % (Aggregate): 70%					

References:

1. *What is Firmware? Architecture and Best Practices*. (2022, October 10). Spiceworks. <https://www.spiceworks.com/tech/devops/articles/what-is-firmware/>
2. Berg, H. K., Rao, P., & Shriver, B. D. (1982). Firmware quality assurance. *National Computer Conference*. <https://doi.org/10.1145/1500774.1500776>
3. *Verifying information flow properties of firmware using symbolic execution*. (2016, March 1). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/abstract/document/7459333/>
4. Al-Hammouri, A. T. (2012). A comprehensive co-simulation platform for cyber-physical systems. *Computer Communications*, 36(1), 8–19. <https://doi.org/10.1016/j.comcom.2012.01.003>
5. Beghi, A., Marcuzzi, F., & Rampazzo, M. (2016). A virtual laboratory for the prototyping of Cyber-Physical systems. *IFAC-PapersOnLine*. <https://doi.org/10.1016/j.ifacol.2016.07.154>
6. *Google Scholar*. (n.d.). https://scholar.google.com/scholar_lookup?title=Cyber-physical%20Systems&publication_year=2011&author=R.%20Baheti&author=H.%20Gil
7. Di Matematica “Tullio Levi-Civita” - Dm, D. (2014). *Computing from LaTeX: automated numerical computing from LaTeX expressions*. <http://paduaresearch.cab.unipd.it/6930/>
8. Beghi, A., Marcuzzi, F., & Rampazzo, M. (2016c). A virtual laboratory for the prototyping of Cyber-Physical systems. *IFAC-PapersOnLine*. <https://doi.org/10.1016/j.ifacol.2016.07.154>
9. Qt Company. (n.d.). *Embedded Software Programming Languages: pros, cons, and comparisons of popular languages*. <https://www.qt.io/embedded-development-talk/embedded-software-programming-languages-pros-cons-and-comparisons-of-popular-languages>
10. *"An Introduction to the Fundamentals of Firmware Engineering for Embedded Systems*. (2023, January). Udemy. https://www.udemy.com/course/firmware-engineering/?amp=&aff_code=Ewh3Y1lWQH8FQR93MkBPbG1RGXFfW1h8B14ZeU5TQ3YBRxFzWj5XMRM%3D&pmtag=CAREERS24LEARN15&utm_source=adwords&utm_medi

um=udemyads&utm_campaign=DSA_Catchall_la.EN_cc.ROW&utm_content=deal4584&utm_term=._ag_88010211481_.ad_535397282064_.kw_.de_c_.dm_.pl_.ti_dsa-

52949608673_.li_1012087_.pd_.&matchtype=&gclid=Cj0KCQjw2qKmBhCfARIsAFy8buIG982c3sLuhWj48XsDI0owJY42DJNT7YhDzSwck8tEbQSaN7Ig5eUaAvBqEALw_wcB

11. *Blog - 4 Communication protocols embedded engineers should know.* (2022, September 28). 5V Media. <https://www.weare5vmedia.com/media/communication-protocols-for-an-embedded-engineer-to-know>
12. *Lab Guide: Embedded Memory Management | Neil's Log Book.* (n.d.). <https://nrqm.ca/nrqm.ca/mechatronics-lab-guide/lab-guide-embedded-memory-management/index.html>
13. Cyrille, C., Dross, C., Gilcher, F., & Moy, Y. (n.d.). *Dynamic Memory Management in Critical Embedded Software.*
14. Beningo, J. (2013, January 10). Building reusable device drivers for microcontrollers. *Embedded.com.* <https://www.embedded.com/building-reusable-device-drivers-for-microcontrollers/>
15. C - *Memory management.* (n.d.). Tutorialspoint. https://www.tutorialspoint.com/cprogramming/c_memory_management.htm
16. GeeksforGeeks. (2022). Memory layout of C programs. *GeeksforGeeks.* <https://www.geeksforgeeks.org/memory-layout-of-c-program/>
17. Gupta, E. (2022, February 28). Memory Layout in C - Scaler topics. *Scaler Topics.* <https://www.scaler.com/topics/c/memory-layout-in-c/>
18. Mathew, H. (2022, October 12). The Complete Guide To Embedded Firmware Development. <https://livepositively.com/>. <https://technocore360.livepositively.com/the-complete-guide-to-embedded-firmware-development/>
19. Xukyo. (2021). Generating and uploading HEX files to an Arduino. *AranaCorp.* <https://www.aranacorp.com/en/generating-and-uploading-hex-files-to-an-arduino/>

- 20.** *Creating a flashable *.hex-file from a C-Source using the arduino tools.* (2017b, August 8). Arduino Forum. <https://forum.arduino.cc/t/creating-a-flashable-hex-file-from-a-c-source-using-the-arduino-tools/473620/11>